Dataset	Domain	Model	R-Tuning	R-Tuning-U	Vanilla-C	Vanilla-U
ParaRel	ID	OpenLLaMA-3B	93.23	**93.33**	88.53	76.96
		LLaMA-7B	93.64	**94.39**	87.92	73.05
		LLaMA-13B	94.44	**95.39**	89.40	79.68
	OOD	OpenLLaMA-3B	69.41	**71.98**	65.54	47.81
		LLaMA-7B	74.61	**76.44**	72.13	48.10
		LLaMA-13B	77.30	**80.87**	69.12	50.52
MMLU	ID	OpenLLaMA-3B	**24.96**	24.60	24.25	21.64
		LLaMA-7B	59.05	**64.69**	48.34	44.00
		LLaMA-13B	**68.87**	66.00	58.69	60.17
	OOD	OpenLLaMA-3B	24.75	**25.52**	23.05	25.26
		LLaMA-7B	**68.69**	67.70	62.79	42.64
		LLaMA-13B	**77.41**	72.66	70.09	64.31

Table 2: Performance comparison of R-Tuning, R-Tuning-U, Vanilla-C, and Vanilla-U with AP scores (%) on the ParaRel and MMLU dataset. Here Vanilla-U denotes evaluating Vanilla-C's answers with R-Tuning-U's sure confidence. ID and OOD denote in-domain and out-of-domain, respectively. The corresponding AP curves are shown in Figure 13.

An advantage of R-Tuning-U is that it does not require the labels of uncertain questions.

5.2 Uncertainty Learning

Uncertainty learning improves AP score. One perspective on interpreting our method is that R-Tuning-U of selecting and learning through uncertainty fundamentally involves learning the uncertainty of the training data. A more direct baseline is to perform vanilla fine-tuning and then use uncertainty selection on the test dataset to respond, a method we refer to as Vanilla-C. Vanilla-C prompts the model to answer k times and choose the majority as the answer. The uncertainty is proportional to the distinct answers. In our experiment, we set $k = 10$ for Vanilla-C and the confidence is calculated by:

$$Confidence = \frac{\max_{i=1}^{n}(k_i)}{k}, \tag{8}$$

where n is the number of distinct answers generated, and k_i is the number of occurrences of i-th answer. We calculate the AP scores and compare Vanilla-C with R-Tuning-U in Table 2. Surprisingly, we find that learning uncertainty and then filtering questions based on this uncertainty to provide answers yields better results than directly filtering and answering questions using uncertainty on the test dataset. In other words, differentiating instruction tuning data based on uncertainty while learning both the correct answers and uncertainty not only enables the learning of uncertainty expressions but also, remarkably, improves the accuracy of question-answering. This is an unexpected but intriguing phenomenon. Learning uncertainty from training data should not be as accurate as using uncertainty estimations directly from the test data. One possible explanation is that for a Transformer model, to accurately predict the last token, the hidden states are adjusted during training. These changes in hidden states might help in better answering easier questions. A potential hypothesis is this: predicting uncertainty embeds information about confidence into the hidden representation. This aids in generating more confident hidden states when answering easier questions. This finding reveals the benefits of learning the uncertainty of large models. It not only avoids the extensive overhead of repeatedly calculating uncertainty during testing but also improves training quality by learning uncertainty, thereby enhancing the accuracy of uncertainty estimation.

Uncertainty learning improves the calibration and prediction. To verify our hypothesis, we conduct further experiments. We first introduce Vanilla-U, which generates the prediction by Vanilla-C and expresses its confidence by R-Tuning-U. Firstly, we find calibration of R-Tuning-U becomes better. We consider the Expected Calibration Error (ECE) metric (Guo et al., 2017), which measures the difference between accuracy and confidence on given confidence intervals. From the Table 16, it is observed that R-Tuning-U improves the calibration, which potentially better indicates answers and improves AP scores. More results are shown in Figures 11, 12. Secondly, from Table 14, we observe that R-Tuning-U improves accuracy compared with Vanilla-C. Furthermore, we also use R-Tuning-U as a scorer to measure the confidence of the answers from both R-Tuning-U and Vanilla-C. The results of Table 15 demonstrate that R-Tuning-U generally receives higher confidence scores than Vanilla-C, which is consistent to the improved accuracy of R-Tuning-U. Finally, Figures 9 and 10 show that score differences become more salient as the models get larger. We conclude that refusal ability is an emergent ability (Wei et al., 2022).

5.3 Unanswerable Questions

In addition to the open-ended question-answering dataset where all the questions are answerable, we also test the performance of R-Tuning on several refusal benchmarks containing unanswerable questions. These questions either contradict common

Dataset	Model	R-Tuning	Vanilla	Pretrain-T
FalseQA	OpenLLaMA-3B	87.32	2.07	9.98
	LLaMA-7B	96.62	18.35	8.92
	LLaMA-13B	95.90	6.00	24.10
NEC	OpenLLaMA-3B	95.72	0.96	7.31
	LLaMA-7B	99.18	20.55	2.02
	LLaMA-13B	98.17	2.36	4.76
SA	OpenLLaMA-3B	90.99	5.23	18.90
	LLaMA-7B	95.45	34.79	16.96
	LLaMA-13B	96.61	12.21	28.00

Table 3: The refusal rate (%) of R-Tuning and other baselines on the refusal benchmarks. SA is the unanswerable part of the SelfAware dataset. The refusal rate of R-Tuning-R on the unanswerable datasets is extremely high, while the refusal rate of other fine-tuned methods and pre-trained models is low.

sense or make up some concepts, and should not receive answers from the model. We verify R-Tuning on such datasets, and the results are shown in Table 3. For baseline models, we provide explicitly in the prompt that they could refuse to answer the questions. We observe that R-Tuning refuses nearly all these unanswerable questions, which meet our expectations, while other baselines answer most of the questions even though they are told to refuse. In conclusion, the R-Tuning possesses the ability to refuse questions that contradict common sense or out of their parametric knowledge.

5.4 Perplexity and Entropy

We further demonstrate the rationale of our method by evaluating the perplexity and the entropy of certain data D_1 and uncertain data D_0. The results are shown in Table 4 and Table 5 respectively. Specifically, we calculate the perplexity of each training question using the pre-trained model to estimate its understanding of them. The lower perplexity of $D1$ shows that the pre-trained model is more familiar with them and is likely to provide correct answers, while the high perplexity of D_0 corresponds to the hallucinations it provides, instead of the correct answers. Besides, larger models generally have a lower perplexity, which explains why they perform better on various tasks.

We also leverage `GPT-3.5-turbo` to answer the questions from D_0 and D_1, and calculate the entropy of the solutions to each question. If `GPT-3.5-turbo` provides multiple solutions to the question, the entropy is relatively high, otherwise it should be low. We observe that the entropy of answers from D_1 is significantly lower than the entropy of D_0, which explains that our method divides the data into two folds. The uncertain data is intrinsically more difficult than certain data. And R-Tuning strategy on D_0 and D_1 teaches the model to answer easy questions with certainty while being conservative in answering difficult questions. More detailed analysis of the perplexity and the entropy are shown in Appendix A.6 and Appendix A.7

Dataset	Model	D_1	D_0
ParaRel	OpenLLaMA-3B	57.92	63.08
	LLaMA-7B	45.81	52.08
	LLaMA-13B	42.79	48.75
MMLU	OpenLLaMA-3B	32.95	462.36
	LLaMA-7B	22.20	115.87
	LLaMA-13B	22.12	81.41
WiCE	OpenLLaMA-3B	61.28	203.43
	LLaMA-7B	20.93	19.40
	LLaMA-13B	17.73	19.56
HotpotQA	OpenLLaMA-3B	144.89	170.38
	LLaMA-7B	49.97	60.19
	LLaMA-13B	42.60	55.20
FEVER	OpenLLaMA-3B	88.38	72.11
	LLaMA-7B	38.46	43.69
	LLaMA-13B	39.00	44.14

Table 4: Perplexity of the training datasets. We run the pre-trained models on the context and questions and calculate the average perplexity.

6 Related Work

In this section, we review the progress on hallucinations of large language models (LLMs) and the uncertainty quantification methods.

6.1 Hallucinations of LLMs

Despite the outstanding performance of LLMs with high fluency and coherence, they are still likely to hallucinate unfaithful and nonfactual facts (Maynez et al., 2020b; Li et al., 2023c). The origin of hallucination is varied. The training data, model training, and model inference processes all have the potential to contribute to hallucination (Zhang et al., 2023c; Ji et al., 2023; Huang et al., 2023b). A large amount of training data may contain misinformation and bias (Dziri et al., 2022; Penedo et al., 2023), leading the model to imitate the falsehood (Lin et al., 2022). Moreover, events evolve over time (Wen et al., 2021; Reddy et al., 2023), and outdated data used for training may contribute to the temporal misalignment problem (Livska et al., 2022; Luu et al., 2022). Additionally, LLMs tend to overestimate their abilities, leading them to sometimes generate incorrect answers with high con-

Dataset	Model	D_1	D_0
ParaRel	OpenLLaMA-3B	0.426	0.709
	LLaMA-7B	0.475	0.694
	LLaMA-13B	0.436	0.744
MMLU	OpenLLaMA-3B	0.347	0.389
	LLaMA-7B	0.330	0.400
	LLaMA-13B	0.239	0.457
WiCE	OpenLLaMA-3B	0.250	0.280
	LLaMA-7B	0.254	0.270
	LLaMA-13B	0.265	0.252
HotpotQA	OpenLLaMA-3B	0.534	0.747
	LLaMA-7B	0.605	0.719
	LLaMA-13B	0.528	0.797
FEVER	OpenLLaMA-3B	0.413	0.219
	LLaMA-7B	0.279	0.286
	LLaMA-13B	0.189	0.350

Table 5: Entropy of the training datasets. It is calculated from the frequency of every predicted answer among all predictions. A larger entropy denotes greater uncertainty of the system.

fidence and fail to identify unknown questions (Yin et al., 2023; Ren et al., 2023; Kadavath et al., 2022). Besides, the alignment with human preference could be problematic, as LLMs may generate responses favoring the users, rather than providing the truth (Perez et al., 2022; Radhakrishnan et al., 2023; Wei et al., 2023b). Moreover, the generation process, including the randomness during inference (Chuang et al., 2023), the snowballing effect to maintain self-consistency with early mistakes (Zhang et al., 2023a), and early local optimization (Azaria and Mitchell, 2023), may also introduce hallucinations.

Recently, a variety of works have been done towards hallucination detection and mitigation. For hallucination detection, Azaria and Mitchell (2023) propose a classifier trained on the internal states of LLMs. Lee et al. (2023) create a benchmark for measuring the factuality of generation, using factual and nonfactual prompts. Manakul et al. (2023) introduce SelfCheckGPT, making use of the consistency of multiple responses from LLM. For hallucination control, retrieval-augmented methods (Peng et al., 2023; Xie et al., 2023; Yue et al., 2023; Lyu et al., 2023; Asai et al., 2023) have shown effectiveness in mitigating the hallucination. Other methods, such as knowledge-aware fine-tuning (Li et al., 2022), corruptions denoising (Chen et al., 2023), low-confidence validation (Varshney et al., 2023), uncertainty-based response ranking (Wan et al., 2024), question-knowledge alignment (Zhang et al., 2023b), knowledge injection and teacher-student model (Elaraby et al., 2023), also improve the factuality of generation from multiple perspectives. Previous studies show the importance of the early discovery of hallucination (Zhang et al., 2023a). In addition, Huang et al. (2023a) found that LLMs cannot rectify themselves with their initial capabilities, displaying the importance of fine-tuning and external feedback. Our proposed method instructs the model to be aware of its knowledge gap between the instruction tuning datasets and the parametric knowledge, so that it possesses the refusal ability when it encounters instructions out of its knowledge.

6.2 Uncertainty Quantification of LLMs

Uncertainty quantification is a long-standing problem in machine learning. In the deep learning era, Guo et al. (2017) first identify the predictive confidence (a.k.a, predictive probability) of deep neural network lack of calibration in terms of the ECE metric (Expected Calibration Error) (Naeini et al., 2015). Chen et al. (2022) further study the investigate the calibration problem of pre-trained large language models and observe the same miscalibration problem on large language models. Active-Prompt (Diao et al., 2023b) introduces uncertainty to select questions for chain-of-thought annotation and demonstrates its effectiveness in actively and judiciously selecting and annotating the most helpful exemplars for in-context learning of LLMs. Knowledge assessment for LLMs (Dong et al., 2023) is also relevant to our study.

7 Conclusion

In this paper, we propose a simple yet effective method, R-Tuning, to teach LLMs to refuse unknown questions. It identifies the difference between instruction tuning data and parametric knowledge and splits the training data into certain and uncertain parts. Then, R-Tuning constructs the refusal-aware data by appending uncertainty expressions to the uncertain part. Empirically, R-Tuning outperforms the traditional finetuning baseline regarding AP score, illustrating a good trade-off between prediction and confidence. R-Tuning not only shows the refusal ability on in-domain data but also demonstrates such ability could be generalized to unseen tasks. It displays that refusal is a fundamental ability and could be abstracted via multi-task learning, so we call it meta-skill.